

CS482/495/496 Software Project Proposal: Baltimore's Basketball Tournament

your name(s) here

2025-10-27

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name:
- Client title:
- Client email address:
- Client employer:
- How you know the client:

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Don't forget to specify them.]

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

We will use an MVC architecture. As we understand them, the "view" is the actual interface of the website, the "model" is the backend or the data that the website runs on, and the "controller" is what connects them, handling input from the UI that needs to be put in the model and translating data from the model to a displayable format in the UI.

The main modules will be for:

- UI design (color, theme, page layout)
- Handling input from the UI (button, form, upload)
- Accessing and modifying database data (image upload/display, account info)

4.2 Diagrams

Not needed for Assignment 2

4.3 Technology

- TypeScript - Programming language
- React - Frontend Library
- Material UI - Frontend Library
- Node.js - Runtime Engine
- Express.js - Backend Framework
- MongoDB - NoSQL Database
- Jest - Testing Framework

4.4 Coding Standards

- Contact the customer (Dr. Ho) if there are any uncertainties with requirements
- We are using standard React naming conventions: PascalCase for components, classes, and interfaces; camelCase for variables and functions; names starting with "use" for custom hooks; and UPPER_CASE for constants. All names should show a clear purpose (just name variables what they do).
- Commit small changes so we can go back if something breaks
- Only code with 60% branch coverage should be committed
- Add tests when bugs are found
- Pair program as much as possible (for learning and bug prevention)
- At least one other team member must accept a merge request
- We will update any documentation necessary as we change the codebase

4.5 Data

The application will utilize MongoDB, which aligns with the MERN stack. The core data structure is organized into several key sections. The users section stores all accounts, differentiating between roles (Admin, Manager, Guardian, Teen) and managing relationships. The tournaments section contains details for each competition, including rules, sponsor logos, and themes. Teams are managed in the teams section, linking them to their respective tournament, manager, and list of player members. Game schedules, scores, and live stream links are stored in the games section, while individual player performances are tracked per game in the playerStats section. Team-based communication is facilitated through the chat section.

The core information collected includes a unique username, email address, password, first and last name, and date of birth. Users will also have the option of uploading a profile picture. The system will assign each user a specific role: administrator, Manager, Guardian, or Teen. To establish critical relationships, the data model includes links between accounts; Guardian accounts will store a reference to the Teen accounts they represent, and Teen accounts will store a reference to their Guardian. Furthermore, user data will track team affiliations, with Managers storing a list of the teams they manage. This forms the key data that will drive our approach. How we form relationships and how we provide information to the users.

4.6 UI Mocks

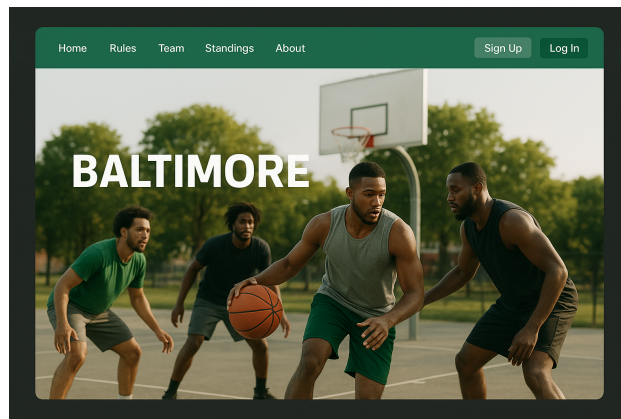


Figure 1: Home Page

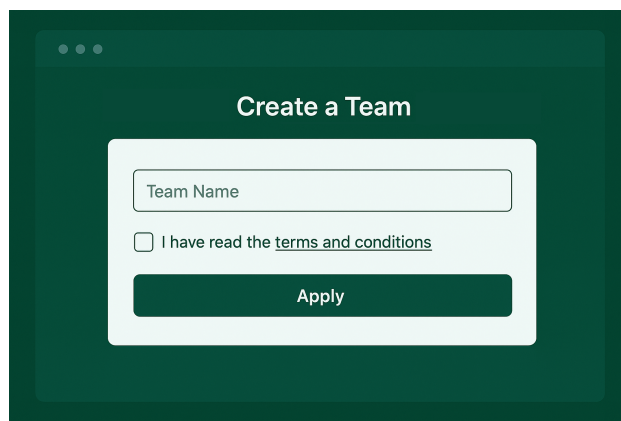


Figure 2: Team Creation

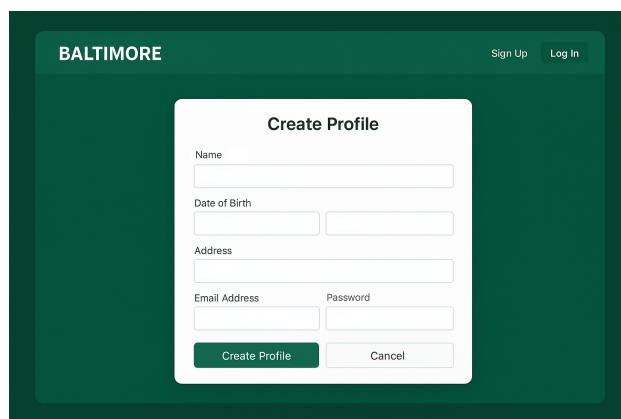
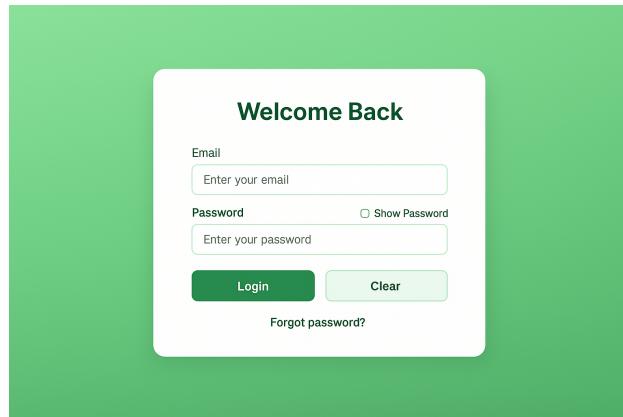


Figure 3: Profile Creation



The login page features a white card centered on a solid green background. The card has a title 'Welcome Back' in bold. Below the title are two input fields: 'Email' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. A 'Show Password' checkbox is located to the right of the password field. At the bottom of the card are two buttons: a green 'Login' button and a light green 'Clear' button. Below the buttons is a link that says 'Forgot password?'.

Welcome Back

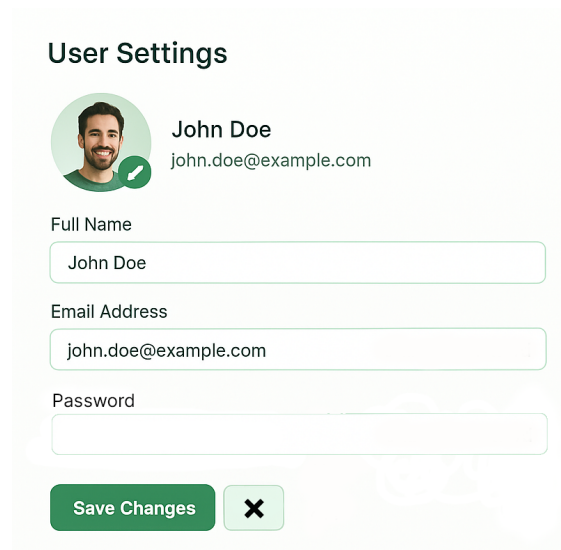
Email
Enter your email

Password ☐ Show Password
Enter your password

Login Clear

[Forgot password?](#)

Figure 4: Login Page



The user settings page has a light green background. At the top is the title 'User Settings'. Below it is a user profile section with a circular avatar of a man, the name 'John Doe', and the email 'john.doe@example.com'. There is a green checkmark icon next to the email. Below the profile section are three input fields: 'Full Name' with the value 'John Doe', 'Email Address' with the value 'john.doe@example.com', and 'Password' which is empty. At the bottom are two buttons: a green 'Save Changes' button and a light green button with a red 'X' icon.

User Settings

 **John Doe**
john.doe@example.com

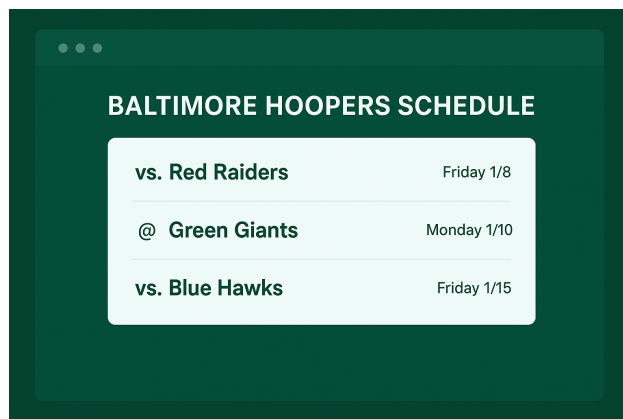
Full Name
John Doe

Email Address
john.doe@example.com

Password

Save Changes X

Figure 5: User Settings



The team schedule page has a dark green background. At the top is the title 'BALTIMORE HOOPERS SCHEDULE'. Below the title is a table with three rows of game information. Each row has a light green background. The first row shows 'vs. Red Raiders' and 'Friday 1/8'. The second row shows '@ Green Giants' and 'Monday 1/10'. The third row shows 'vs. Blue Hawks' and 'Friday 1/15'.

BALTIMORE HOOPERS SCHEDULE	
vs. Red Raiders	Friday 1/8
@ Green Giants	Monday 1/10
vs. Blue Hawks	Friday 1/15

Figure 6: Team Schedule

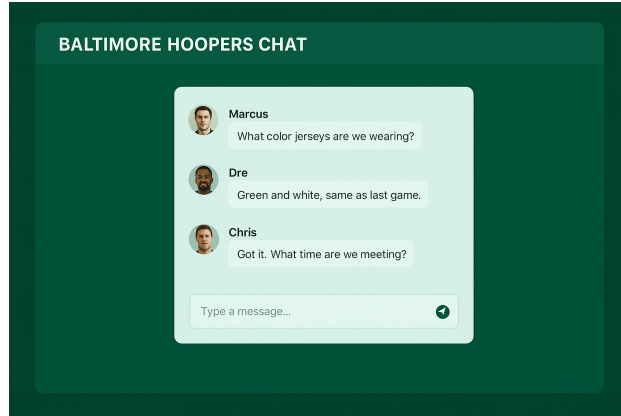


Figure 7: Team Chat Room

5 Iterations

5.1 Iteration Planning

~~[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]~~

Iteration	Dates	Stories	Points
1	10/09 - 10/23	T2, A11, M1, U17, Spike20, A14, V5, U18	16
Total:			16

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

~~[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]~~

Overall, we wanted to focus on the basics of the application, which included creating an account, creating a team, joining a team, and setting up sponsor programs. These user stories would lead to us creating the basics of the application, and get a good handle on how our codebase would look like.

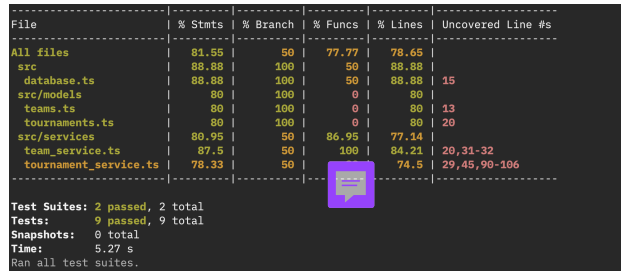
5.2.2 Work Done

~~[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]~~

Manuel worked on the tournament creation/reading and team creation/reading. He was able to setup the basic schema of the two, and the routes for the endpoints required to create/read all/read one. However, he did not yet include authentication, as only parent accounts should be able to create teams, and only admin accounts should be able to create tournaments.

5.2.3 Testing Coverage

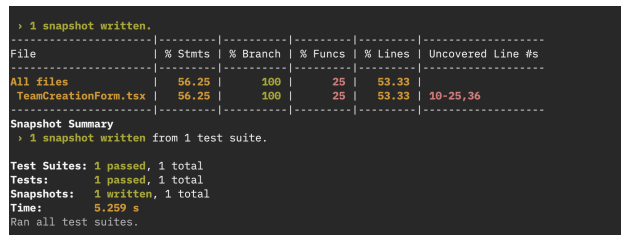
~~[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]~~



File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	81.85	50	77.77	78.65	
src	88.88	100	50	88.88	
database.ts	88.88	100	50	88.88	15
src/models	80	100	0	80	
teams.ts	80	100	0	80	13
tournaments.ts	80	100	0	80	20
src/services	80.95	50	86.95	77.14	
team_service.ts	87.5	50	100	84.21	20,31-32
tournament_service.ts	78.33	50		74.5	29,45,98-106

Test Suites: 2 passed, 2 total
 Tests: 9 passed, 9 total
 Snapshots: 0 total
 Time: 5.27 s
 Ran all test suites.

Figure 8: Back-end Coverage



File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	56.25	100	25	53.33	
TeamCreationForm.tsx	56.25	100	25	53.33	10-25,36

Snapshot Summary
 1 snapshot written from 1 test suite.

Test Suites: 1 passed, 1 total
 Tests: 1 passed, 1 total
 Snapshots: 1 written, 1 total
 Time: 5.259 s
 Ran all test suites.

Figure 9: Front-end Coverage

Overall, we should definitely increase both our front-end and back-end coverage.

5.2.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

This iteration, a lot of our individual time went towards a) learning the basics of web-development and b) setting up the code-base. Many of our group members were unfamiliar with all of the parts of web-development, which required time outside of working on the project to tinker around with tutorials and guides unrelated to this project specifically. All of this required learning caused a lot of friction before we could get to actual programming. However, we all learned a lot about the specific libraries we're using. Now that we have a foundation, the next iteration should go smoother.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some

artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.3.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482)]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

Appendix

[Appendix section if needed]